WAREHOUSE STOCK MANAGER

Static Analysis & Remediation Report | 23 October 2026

Scope and methodology

Three baseline scans were supplied for the original Python file. This report maps their findings to changes in **warehouse_stock_manager_fixed.py**, then records the independent verification performed on the revised code. The original reports are not mistaken for new scan output.

Results at a glance

Tool / evidence	Original baseline	Revised state
Pylint	37 messages; 4.48/10	Changes implemented; re-scan pending
Flake8	19 violations	Style patterns corrected; re-scan pending
Bandit	5 findings (1 high, 2 medium, 2 low)	All 5 mapped to mitigations; re-scan pending
Behavioral regression tests	No test output supplied	19/19 passed
Python compilation	No output supplied	Passed

Interpretation

The code has been revised to address every reported category. **No post-fix Pylint score or actual post-fix Flake8/Bandit count is claimed.** The three tools could not be installed in the execution environment because package-index connectivity was unavailable.

Key security and logic improvements

- Removed execution of arbitrary Python expressions via `eval()`; only limited AST-based integer operations are permitted.
- Supervisor password is obtained from `WAREHOUSE_SUPERVISOR_PASSWORD` instead of source code.
- Stock operations reject negative inventory, invalid SKU types, and failed removals.
- File processing uses UTF-8 context managers; JSON loading validates data before replacing inventory.

Test case 01 | Pylint

Purpose: identify maintainability and code-quality warnings. Baseline: **37 reported messages; score 4.48/10**.

Baseline code	Count	Implemented change
C0116 - missing function docstrings	10	Added concise function documentation
C0103 - non-snake_case names	9	Renamed operations to snake_case
W1514 - unspecified file encoding	3	Explicit UTF-8 file reads/writes
R1732 - manual file management	3	Replaced with context managers
W0123 - eval used	2	Restricted AST arithmetic; plain print
W0611 - unused imports	2	Cleaned up imports (os is now needed)
Other findings	8	Whitespace, unused locals, defaults, global rebinding, exceptions, f-strings, documentation

Notable before/after: `addStock(..., history=[])` becomes `add_stock(..., history=None)`; `loadStock` now validates data and updates the existing dictionary without a `global` rebinding.

Result: Source-level fixes are visible. A final Pylint score requires running Pylint on the revised module.

Test case 02 | Flake8

Purpose: identify formatting and coding-style violations. Baseline: **19 reported violations**.

Rule	Count	Correction
E302	10	Separated top-level functions with two blank lines
E501	3	Wrapped overlong statements
F401	2	Eliminated unused imports
E722	1	Removed bare except
F841	1	Removed unused total_units variable
W291	1	Removed trailing whitespace
E305	1	Corrected blank lines after main definition

Independent source checks: no lines exceeding 79 characters, no trailing whitespace, and successful Python compilation. These targeted checks are not a replacement for a Flake8 re-scan.

Test case 03 | Bandit

Purpose: identify unsafe coding patterns and potential vulnerabilities. Baseline: **5 findings** (1 high, 2 medium, 2 low).

Finding	Severity	Original problem	Revised mitigation
B105	Low	Hardcoded password	Use environment configuration and constant-time comparison
B110	Low	Silent try/except/pass	Explicit KeyError and ValueError for stock failures
B324	High	MD5 SKU fingerprint	Truncated SHA-256 label identifier
B307	Medium	eval on supervisor formula	AST allowlist: +, -, *, //, unary +/-
B307	Medium	eval used for print	Direct print statement

Independent AST checks: no direct eval calls, no calls to MD5, and no bare exception handlers were found in the revised module.

Security qualification: The six-character SHA-256 fingerprint is suitable only as a short label marker, not authentication or tamper detection. Passing Bandit would not establish comprehensive application security.

Additional functional safeguards

- Validate non-empty string SKUs and positive bounded integer stock inputs.
- Prevent over-removal, overflow, and negative inventory.
- Reject invalid JSON stock values before changing current inventory.
- Write structured JSON audit events for authorized adjustments.
- Avoid side effects when the module is imported by guarding main().
- Require environment configuration for supervisor adjustment; demo skips it if unset.

Compatibility considerations

Public functions were renamed (for example, `removeStock` to `remove_stock`), so dependent scripts must update calls. Supervisor adjustment now requires an environment variable and does not use the previous hardcoded credential.

Independent verification

The revised module was compiled and tested locally. These results are separate from the three original static-analysis reports.

Verification	Measured outcome
19 behavioral regression tests	PASS - all 19 tests
Python syntax compilation	PASS
Direct eval() calls	0 identified
hashlib.md5() calls	0 identified
Bare exception handlers	0 identified
Lines over 79 characters	0 identified
Trailing-whitespace lines	0 identified
Pylint / Flake8 / Bandit post-fix scans	NOT RUN - scanner installation unavailable

Regression cases included stock addition and retrieval; SKU and quantity validation; capacity limits; missing/insufficient-stock removal; reorder threshold; checksum; JSON round-trip and invalid-load handling; accepted and rejected adjustment formulas; unauthorized access; audit-event creation; and safe importing.

Reproduce the three scans

```
python -m pip install pylint flake8 bandit
python -m py_compile warehouse_stock_manager_fixed.py
python -m unittest -v test_warehouse_stock_manager_fixed.py
pylint warehouse_stock_manager_fixed.py > pylint_after.txt
flake8 warehouse_stock_manager_fixed.py > flake8_after.txt
bandit -f txt warehouse_stock_manager_fixed.py > bandit_after.txt
```

Remaining scope / production considerations

- Store operational secrets and audit logs securely; environment configuration alone does not provide a full credential-management policy.
- For concurrent updates and crash-safe persistence, use transactions or atomic writes; this prototype uses in-memory state and JSON files.
- Obtain actual post-fix outputs from all three scanners before declaring a completely clean result or a new Pylint rating.

Conclusion

The supplied baseline documented **37 Pylint messages (4.48/10)**, **19 Flake8 violations**, and **5 Bandit security findings**. The revised code implements corresponding fixes and passes **19/19 regression tests**, compilation, and focused static inspection. Formal acceptance should include a new Pylint, Flake8, and Bandit scan on the exact revised file.

Sources: pylint_report.txt; flake8_report.txt; bandit_report.txt; original and revised Python modules; verification_tests.txt; verification_static_checks.json.

Reflection questions and answers

Responses reflect the three supplied reports, the revised code, and local verification. They do not imply that the static analyzers were re-run.

Q1. Why were Pylint, Flake8, and Bandit all needed?

Answer: Each tool examines a different aspect of the code. Pylint highlighted maintainability and programming practices, Flake8 focused on style and simple errors, and Bandit identified potentially unsafe patterns. Using all three gave a broader picture than relying on one scan.

Q2. Which original issue was most concerning, and why?

Answer: The two uses of eval() were especially concerning because an expression could be treated as executable Python code. Although the supplied Bandit report classified MD5 as the single high-severity finding, the eval() paths represented important execution risks and needed to be removed.

Q3. Why was eval() replaced with restricted AST-based arithmetic?

Answer: The supervisor adjustment feature still needed calculations such as current + 25. A controlled expression parser can allow specific numeric operations while rejecting function calls, attribute access, and unrelated Python statements. This preserves the intended feature without unrestricted code execution.

Q4. What did the Pylint findings teach about maintainability?

Answer: I learned that names, documentation, resource management, and safe defaults affect how easily code can be understood and extended. Changing addStock to add_stock, adding docstrings, using with open(), and replacing history=[] with history=None address maintainability problems rather than merely changing appearance.

Q5. Why should Flake8 issues be fixed if the program already runs?

Answer: The original 19 Flake8 findings included spacing, long lines, unused imports, an unused variable, and a bare except. Cleaning these makes reviews more consistent and removes distracting warnings. It improves clarity, but it does not by itself establish that business logic or security is correct.

Reflection questions and answers | continued

Q6. How was hidden or incorrect stock behavior improved?

Answer: The earlier removeStock operation silently ignored errors, while the example could introduce negative quantities or an invalid SKU type. The revised code validates SKUs and quantities, rejects missing or insufficient stock, and checks loaded JSON before updating the inventory.

Q7. What was learned about secrets and checksums?

Answer: A supervisor password should not be embedded in source code; the revised version reads it from an environment variable. MD5 was replaced with a truncated SHA-256 fingerprint for labels, but the short identifier is not suitable as proof of authenticity or protection against tampering.

Q8. Do the 19 passing regression tests prove that all three scans now pass?

Answer: No. The 19 tests, successful compilation, and focused source checks confirm specific behaviors and patterns in the revised implementation. Pylint, Flake8, and Bandit were not re-run on that file, so a new Pylint score and post-fix finding totals must not be claimed.

Q9. What compatibility trade-offs resulted from the fixes?

Answer: Renaming camelCase functions to snake_case means existing callers need updates. Supervisor adjustments also depend on a configured environment variable rather than the old hardcoded password. Such changes improve code quality and security but require integration updates and documentation.

Q10. What should be done before calling this project complete?

Answer: Run Pylint, Flake8, and Bandit against the exact corrected file and archive the outputs. Add more edge-case and integration tests, then consider secure audit-log storage, atomic writes, and concurrency controls before using the program beyond a small single-process prototype.